ORY SEGAL
CO-FOUNDER AND CTO OF PURESEC

Secure Serverless CI/CD with Codeship, PureSec, and AWS Lambda

About the Author.

Ory is a world-renowned expert and veteran in application security with 20 years of experience. He's Founder & CTO of PureSec. Before PureSec he was Senior Director of Threat Research at Akamai.

Codeship is a fully customizable hosted Continuous Integration and Delivery platform that helps you build, test, and deploy web applications fast and with confidence.

Learn more about Codeship here.

Share this Codeship Guide

Secure Serverless CI/CD with Codeship, PureSec, and AWS Lambda

When you develop, build, and deploy applications using a serverless architecture, you have no access to the infrastructure on which traditional security controls used to exist.

This means that you cannot deploy any traditional security protections such as firewalls, intrusion prevention, web application firewalls (WAF), threat prevention, or even runtime application self-protection (RASP) solutions.

Nonetheless, serverless applications may contain application layer vulnerabilities, which may enable a malicious user to mount application layer attacks.

PureSec provides an end-to-end serverless security solution and is the only Security Partner of AWS Lambda.

ABOUT THIS EBOOK

In this eBook we will talk about Secure Serverless CI/CD with Codeship, PureSec, and AWS Lambda.

Malicious users could also tamper with or leak sensitive data by manipulating serverless function logic through business logic attacks.

The PureSec solution can statically scan serverless projects for known vulnerabilities, over-permissive IAM roles, and security policies and insecure application secrets storage. The purpose of the static analysis component of the PureSec solution is to make sure that organizations build and deploy robust serverless applications.

In addition to serverless project hardening, the PureSec solution also provides a unique runtime application layer protection, which is capable of accurately detecting attacks or malicious behaviors and blocking them in real time.

Using a simple five-minute integration between the PureSec serverless security solution and Codeship's cloud CI/CD solution, customers can quickly and painlessly build, deploy, and test serverless applications with security baked in from the get-go.

PureSec SSRE Technology

PureSec's SSRE (Serverless Security Runtime Environment) provides a trusted and safe computing environment for secure serverless function operation, with the following benefits:

- **Protocol agnostic:** SSRE protects serverless functions from attacks coming in from any type of serverless event triggers such as:
  - NoSQL events (eg, AWS DynamoDB)
  - Stream processing events (eg, AWS Kinesis)
  - Code Repository changes (eg, GitHub)
  - Message queue events (eg, AWS SNS)
  - Cloud Storage events (eg, Azure Blob Storage)
  - HTTP APIs
  - Emails
  - SMS messages
  - IoT telemetry events (eg, MQTT or WebSockets)

- **Supports all runtime languages:** PureSec SSRE provides protections for serverless functions written in all major supported language such as:
  - Java
  - Python
  - JavaScript (Node.js)
  - and in the near future, support for Golang, C#, and other languages

- **Cloud vendor agnostic:** SSRE technology can protect serverless functions on any FaaS platform, with current support provided for AWS Lambda and Azure Functions.

The PureSec SSRE provides full-stack, end-to-end serverless security protection for your applications.

Serverless application firewall

This feature provides a deep application layer inspection of event-trigger data; it detects the most sophisticated attack payloads consumed from any type of serverless events.

Serverless behavioral protection

This unique behavioral protection for serverless function execution blocks 0-day exploits and unknown attack vectors. It creates a trusted and secure computing environment for the function to execute in.

PureSec SSRE detects and blocks any attempt to subvert serverless function logic. For example, it blocks:

- attempts to leak data
- unauthorized outbound traffic
- attempts to download and execute malware
- malicious code from executing as part of the function's normal execution
- unauthorized access to files

Serverless environment hardening

From build to deployment, SSRE integrates into your CI/CD process seamlessly. During build time, serverless projects are statically scanned to pinpoint risks related to over-permissive IAM roles and access rights.

The CI/CD scanner also detects insecure storage of application secrets and flags known vulnerable third-party dependencies. With the CI/CD integration, customers can decide to fail a build that does not meet corporate security policy.

Unparalleled serverless observability and monitoring

SSRE provides unparalleled deep monitoring of your serverless functions. Always keep an eye on your functions behavior in real time, including function inventory, invocation data, runtime behavior, event data, security alerts, and threat intelligence.

CHESLEY BROWN, INVISION APP

From a vision perspective, Codeship's Continuous Integration service has hit the nail on the head.

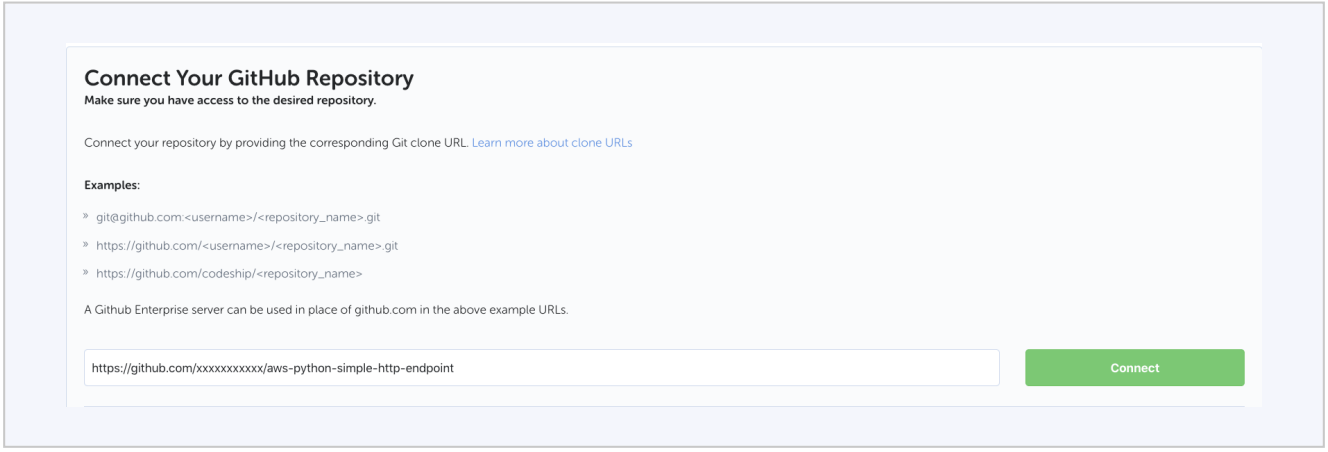
LEARN MORE

Configuring Your Serverless CI/CD With Codeship and PureSec

In our sample project, we will use the Codeship GitHub SCM integration and trigger the build once our GitHub project has changed.

As an example, we will use the Serverless "examples" project titled `aws-python-simple-http-endpoint`, which you can download from the Serverless examples GitHub repository.

First, make sure you clone this repository. Then go on to configure CodeShip to connect to your GitHub repository as follows:



Next, we will configure our project with the following setup commands (line wrapped):

CODE

```
nvm install 8.10
npm install -g serverless
mkdir lib
cd lib
pip install puresec-awslambda --extra-index-url=http
ps://**********************************:@packagecloud.io/puresec/puresec-awslambda/
pypi/simple -t .
cd ../
pip3 install puresec-cli-enterprise --extra-index-url=https://********.puresec.
io/************************/puresec-cli-enterprise/python
pyenv local 3.6
puresec test-roles --fail-findings 1
serverless deploy
```

The build steps above perform the following:

- Update the Node runtime to the latest, required for installing the serverless framework.
- Install the serverless framework.
- Create a /lib/ directory in your project, which will host the PureSec protection library.
- Install the PureSec protection library.

- Install the PureSec CI/CD static analyzer.
- Perform the static analysis of your serverless project, while instructing PureSec to fail the build in case any vulnerabilities were detected — this will make sure that you only deploy hardened and robust serverless functions.
- In case everything succeeds, the build will continue to deploy your AWS Lambda function in your respective AWS account.

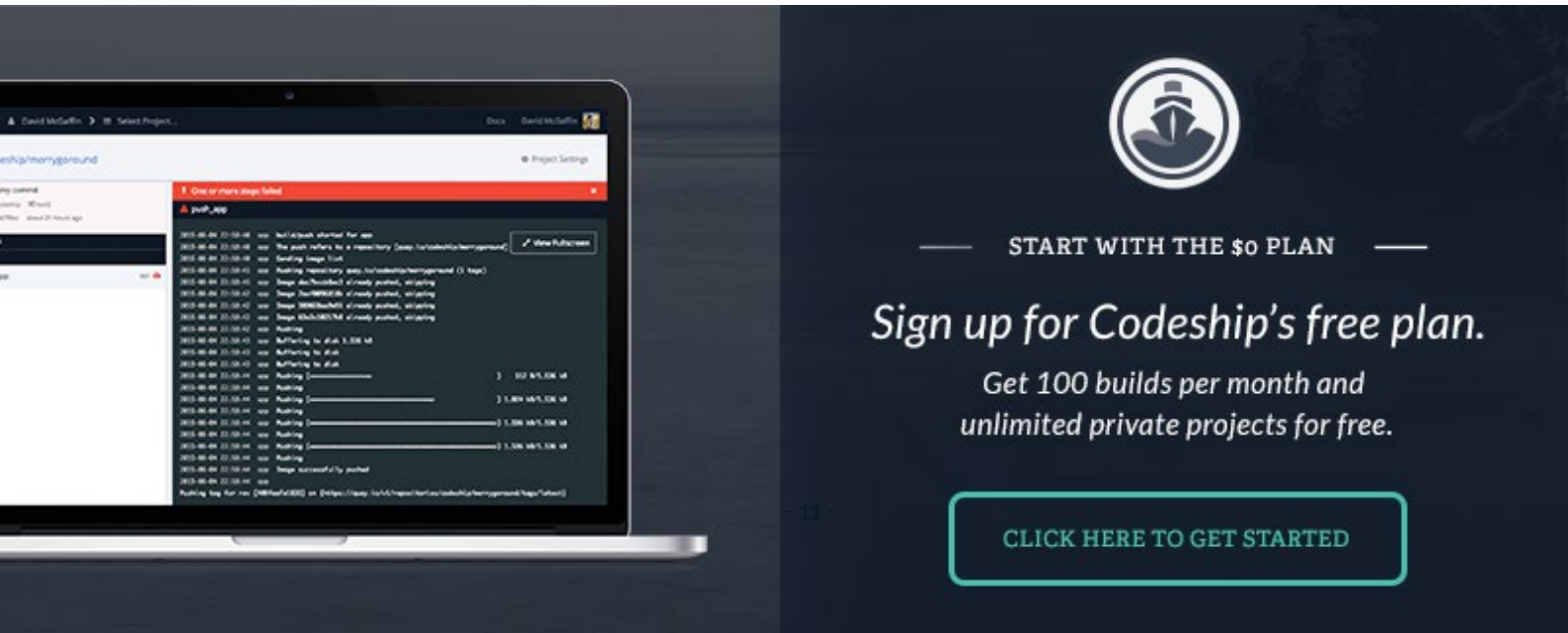
Next, we will configure the following test pipeline:

CODE

```
serverless invoke -f currentTime -d "{\"username\":\"foobar\"}" -l | tee legitimate.txt
serverless invoke -f currentTime -d "{\"username\":\"foobar' UNION SELECT password from tblUsers;--\"}" -l | tee attack.txt
grep "current time" legitimate.txt
grep "SQLi" attack.txt
```

The test pipeline above includes the following steps:

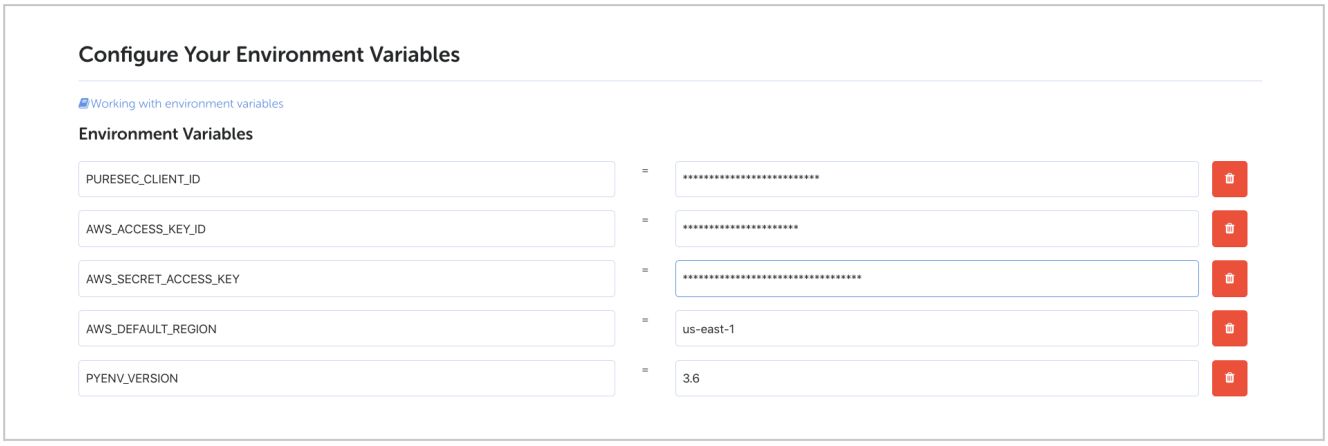
- Invoke your serverless function with legitimate input in order to verify that the function was deployed and works properly.
- Invoke your serverless function with a sample SQL injection attack payload, which should trigger the PureSec serverless application firewall. If the test succeeds, this means that your serverless function is now protected by PureSec. If this fails, you probably forgot to secure your function with PureSec.
  - Success criteria is defined as follows: if the function returns the string "current time" it means it worked properly.
  - Fail criteria is defined as follows: if the function returns the string "SQLi", it means that PureSec caught the attack and raised a security exception with the attack_type = SQLi



In order to be able to deploy and test our AWS Lambda function, we will need to configure the following environment variables:

CODE

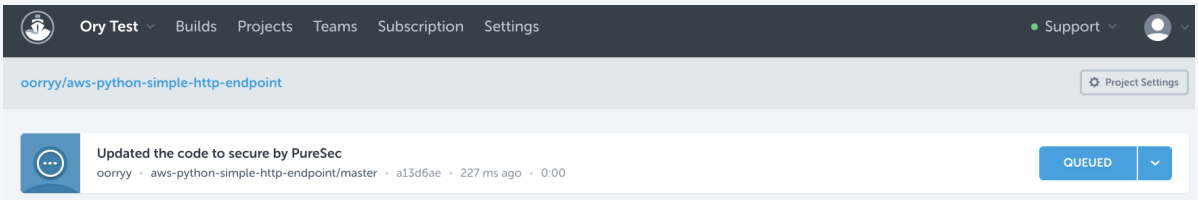
```
PURESEC_CLIENT_ID = (your PureSec Client ID)
AWS_ACCESS_KEY_ID = (your AWS access key ID)
AWS_SECRET_ACCESS_KEY = (your AWS secret access key)
AWS_DEFAULT_REGION = (your AWS deployment region)
PYENV_VERSION = 3.6
```



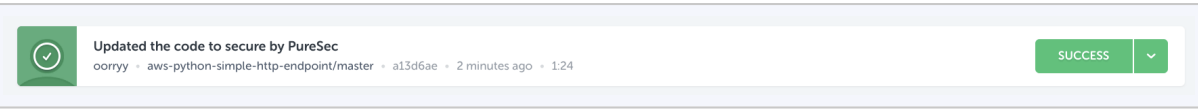
The final step is to actually secure our AWS Lambda function with PureSec. This is done through adding three lines of code (lines 6, 7, and 9 in the screenshot below) in the beginning of our function.

```python
import json
import datetime
import sys

# Add PureSec Security
sys.path.append("lib")
import puresec

@puresec.handler("Unauthorized execution!", skip=("user.address",))
def handler(event, context):
    current_time = datetime.datetime.now().time()
    body = {
        "message": "Hello, the current time is " + str(current_time)
    }

    response = {
        "statusCode": 200,
        "body": json.dumps(body)
    }
    return response
```

That's it – all we have to do now is push some changes to our project, and watch how Codeship does its thing:



Our build finished successfully.



Here's how the final steps of the test pipeline look in the dashboard:

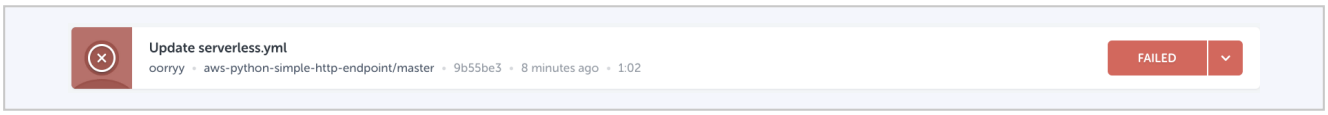
```
16 serverless deploy                                                                    00:25
17 serverless invoke -f currentTime -d "{\"username\":\"foobar\"}" -l | tee legitimate.txt   00:03
{
"statusCode": 200,
"body": "{\"message\": \"Hello, the current time is 12:13:59.110136\"}"
}
--------------------------------------------------------------------------------
START RequestId: c3d98bb1-6339-11e8-b156-7343f2d05ef3 Version: $LATEST
END RequestId: c3d98bb1-6339-11e8-b156-7343f2d05ef3
REPORT RequestId: c3d98bb1-6339-11e8-b156-7343f2d05ef3  Duration: 10.48 ms      Billed Duration: 100 ms         Memory Size: 1024 MB    Max Memory Used: 36 MB
18 serverless invoke -f currentTime -d "{\"username\":\"foobar' UNION SELECT password from tblUsers;--\"}" -l | tee attack.txt   00:03
{
"statusCode": 200,
"body": "{\"message\": \"Hello, the current time is 12:14:00.757657\"}"
}
--------------------------------------------------------------------------------
START RequestId: c532dad7-6339-11e8-9131-0fac93562c14 Version: $LATEST
2018-05-29 12:14:00.758 (+00:00)        c532dad7-6339-11e8-9131-0fac93562c14    [INFO]  {"puresec_log": true, "puresec_customer_log_identifier": null, "puresec_client_id": null,
"module": "PythonFirewall", "metapackage_version": "0.0.23", "invoked_function_arn": "arn:aws:lambda:us-east-1:998167480069:function:aws-python-simple-http-endpoint-dev-currentTime",
"function_version": "$LATEST", "aws_request_id": "c532dad7-6339-11e8-9131-0fac93562c14", "log_type": "alert", "mode": "alert", "data": {"malicious_input": "foobar' UNION SELECT
password from tblUsers;--\n", "event": {"username": "foobar' UNION SELECT password from tblUsers;--"}, "alert_type": "[SQLi]"}}
2018-05-29 12:14:00.758 (+00:00)        c532dad7-6339-11e8-9131-0fac93562c14    [INFO]  {"puresec_log": true, "puresec_customer_log_identifier": null, "puresec_client_id": null,
"module": "AWSPythonAnalytics", "metapackage_version": "0.0.23", "invoked_function_arn": "arn:aws:lambda:us-east-1:998167480069:function:aws-python-simple-http-endpoint-dev-
currentTime", "function_version": "$LATEST", "aws_request_id": "c532dad7-6339-11e8-9131-0fac93562c14", "log_type": "analytics", "mode": "alert", "data": {"container_id": "81789119-
bc71-473d-bfcb-847893e3815a", "host_name": "ip-10-17-78-133\n", "region": "us-east-1", "runtime": "AWS_Lambda_python3.6", "version": "$LATEST", "name": "aws-python-simple-http-
endpoint-dev-currentTime", "memory": "1024", "behavior_load_time": 74, "memory_limit_in_mb": "1024", "aws_request_id": "c532dad7-6339-11e8-9131-0fac93562c14",
"firewall_execution_time": 7, "function_executions": 2}}
END RequestId: c532dad7-6339-11e8-9131-0fac93562c14
REPORT RequestId: c532dad7-6339-11e8-9131-0fac93562c14  Duration: 8.11 ms       Billed Duration: 100 ms         Memory Size: 1024 MB    Max Memory Used: 37 MB
19 grep "current time" legitimate.txt                                                   00:01
20 grep "SQLi" attack.txt                                                               00:01
```

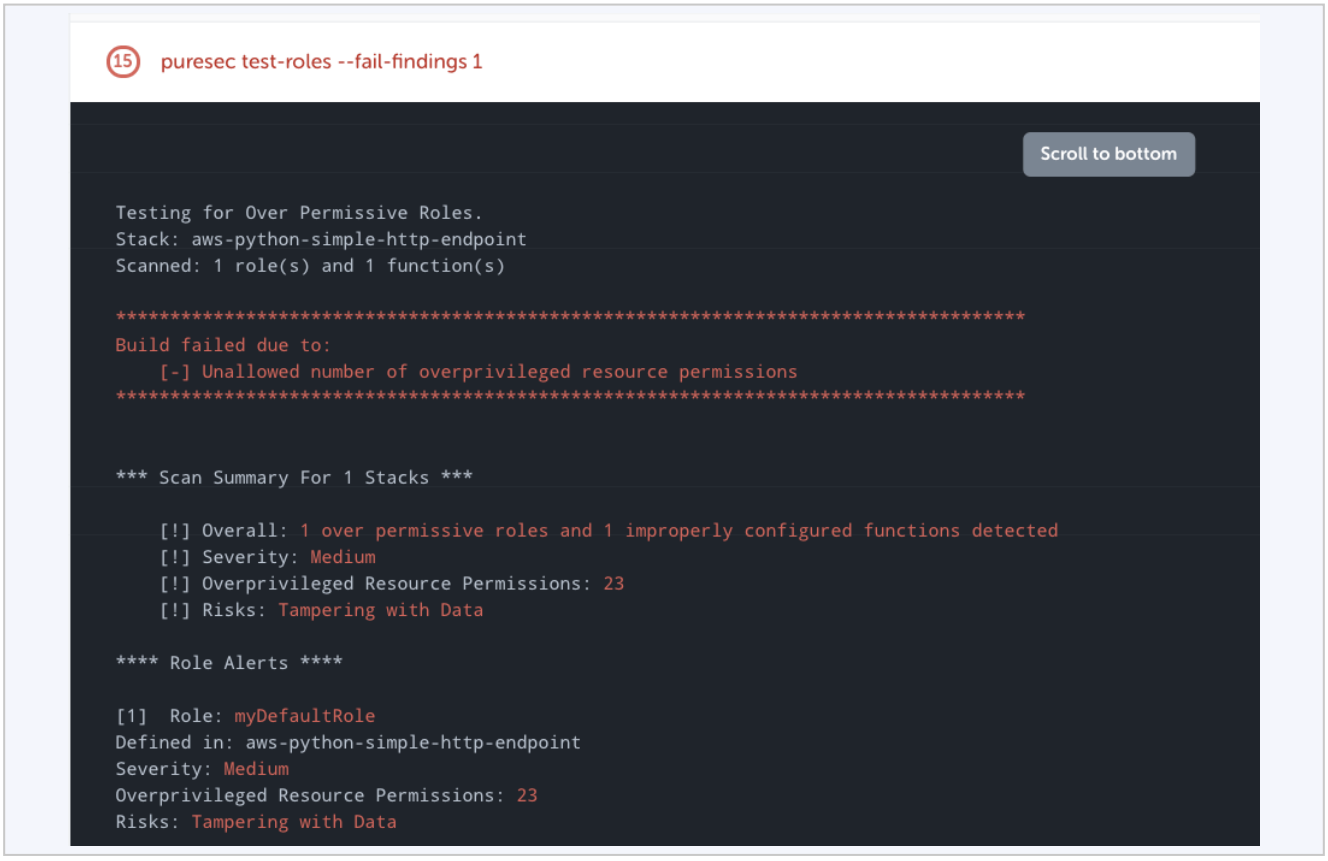
Now, let's deploy an insecure function, which contains an over-permissive IAM security policy. It will grant the function unnecessary `PutObject` permissions on all S3 buckets in the AWS account:

CODE

```
- Effect: Allow
  Action:
  - "s3:PutObject"
  Resource: "arn:aws:s3:*"
```

The build should fail and return the following security vulnerabilities:

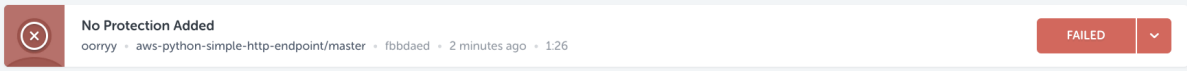




PureSec also goes the extra step and provides insights into the risks involved with these insecure permissions:

```
**** Role Alerts ****

[1]  Role: myDefaultRole
Defined in: aws-python-simple-http-endpoint
Severity: Medium
Overprivileged Resource Permissions: 23
Risks: Tampering with Data

Over Permissive Functions:
Stack: aws-python-simple-http-endpoint
1. Function Name: currentTime
[!] [H,WR,TAM] s3:PutObject                    -> aws-python-simple-http-e-serverlessdeploymentbuck-xafxzph4zjzl
[!] [H,WR,TAM] s3:PutObject                    -> aws-python-simple-http-endpoint-ServerlessDeploymentBucket
[!] [H,WR,TAM] s3:PutObject                    -> aws-python3-dev-serverlessdeploymentbucket-1goos1do2q4lr
[!] [H,WR,TAM] s3:PutObject                    -> collector-dev-serverlessdeploymentbucket-8g9gvos3owp1
```

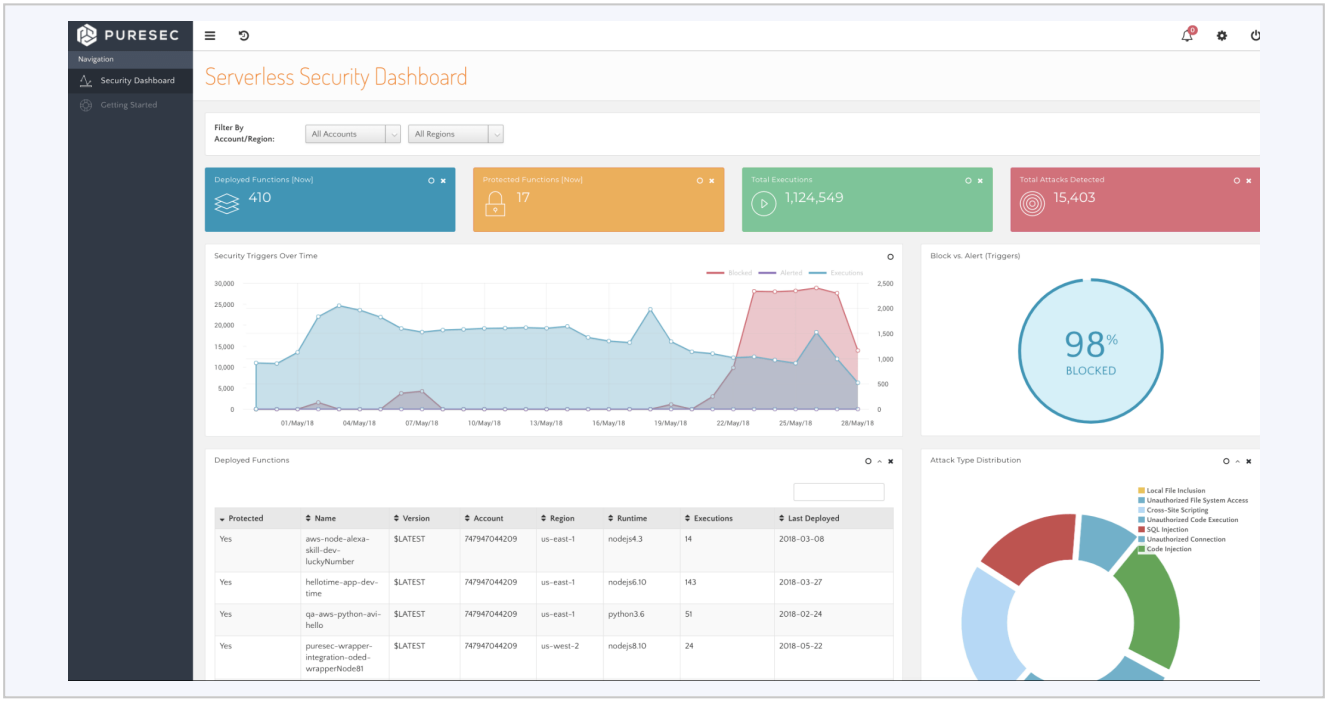
Now, let's deploy our function with the proper IAM roles and permissions. However, we will "forget" to include the PureSec runtime protection library as a dependency, and see what happens to our build.



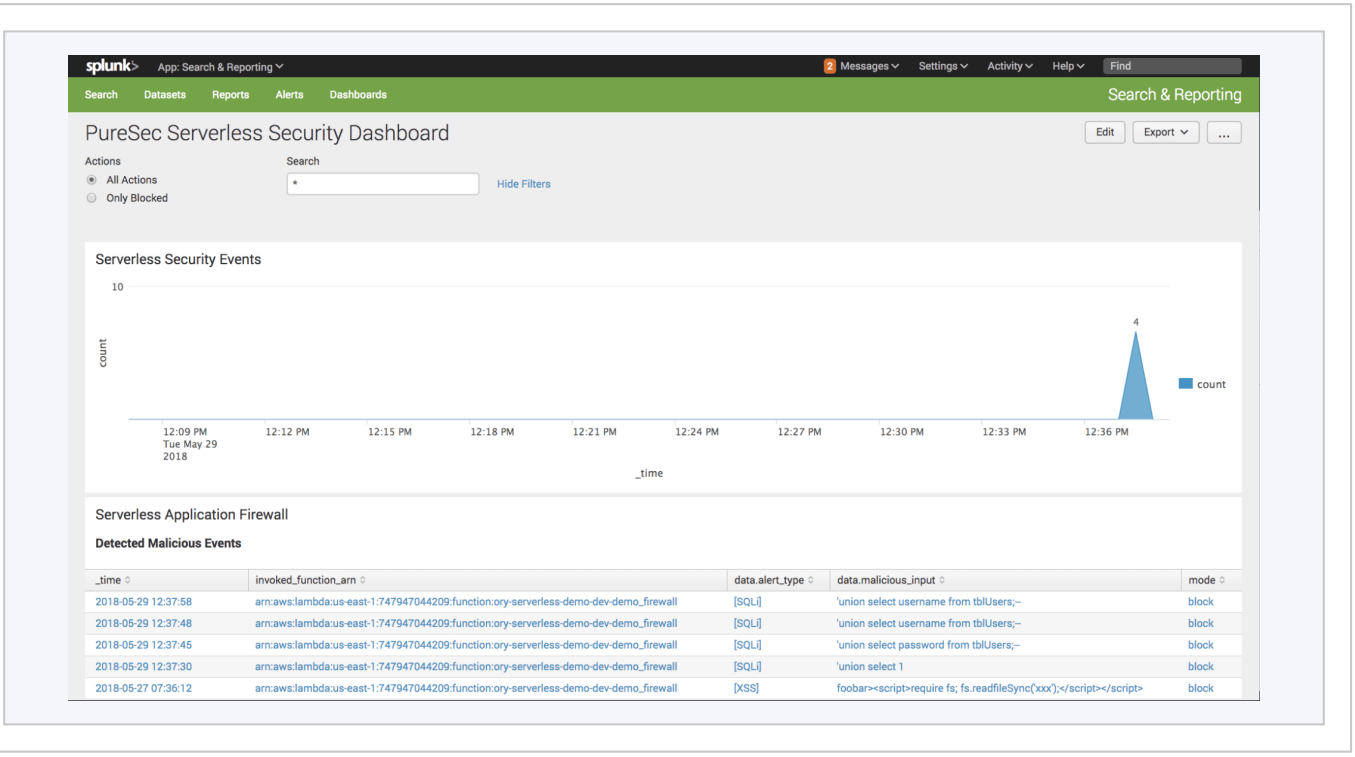
Looking at the build output, we can see that our sample SQL injection attack managed to reach our application, and no protection was applied.

```
17  serverless invoke -f currentTime -d "{\"username\":\"foobar\"}" -l | tee legitimate.txt          00:04
18  serverless invoke -f currentTime -d "{\"username\":\"foobar' UNION SELECT password from tblUsers;--\"}" -l | tee attack.txt   00:02
19  grep "current time" legitimate.txt                                                              00:02
20  grep "SQLi" attack.txt                                                                          00:01
```

After the deployment is done properly, PureSec provides unparalleled visibility into your serverless security posture through its SaaS dashboard. This not only flags attacks in near real-time but also provides a cross-account/cross-region inventory of all your deployed serverless functions, their version, runtime information, and whether or not they are secured by PureSec.



Customers may also integrate the PureSec solution with Splunk in order to analyze and perform security forensics when something bad happens.


splunk> App: Search & Reporting
Messages
Settings
Activity
Help
Find
Search
Datasets
Reports
Alerts
Dashboards
Search & Reporting
PureSec Serverless Security Dashboard
Edit
Export
...
Actions
All Actions
Only Blocked
Search
*
Hide Filters
Serverless Security Events
10
count
4
count
12:09 PM
Tue May 29
2018
12:12 PM
12:15 PM
12:18 PM
12:21 PM
12:24 PM
12:27 PM
12:30 PM
12:33 PM
12:36 PM
_time
Serverless Application Firewall
Detected Malicious Events
_time
invoked_function_arn
data.alert_type
data.malicious_input
mode
2018-05-29 12:37:58
arn:aws:lambda:us-east-1:747947044209:function:ory-serverless-demo-dev-demo_firewall
[SQLi]
'union select username from tblUsers;--
block
2018-05-29 12:37:48
arn:aws:lambda:us-east-1:747947044209:function:ory-serverless-demo-dev-demo_firewall
[SQLi]
'union select username from tblUsers;--
block
2018-05-29 12:37:45
arn:aws:lambda:us-east-1:747947044209:function:ory-serverless-demo-dev-demo_firewall
[SQLi]
'union select password from tblUsers;--
block
2018-05-29 12:37:30
arn:aws:lambda:us-east-1:747947044209:function:ory-serverless-demo-dev-demo_firewall
[SQLi]
'union select 1
block
2018-05-27 07:36:12
arn:aws:lambda:us-east-1:747947044209:function:ory-serverless-demo-dev-demo_firewall
[XSS]
foobar><script>require fs; fs.readfileSync('xxx');</script></script>
block

Summary

Serverless applications can easily become the target of application layer attacks, which can lead to devastating outcomes.

Prior to building and deploying serverless projects, you should make sure that your serverless applications are scanned for serverless layer vulnerabilities, insecure IAM permissions, and known issues. In addition, serverless functions should always be protected using a serverless security runtime environment solution, such as the PureSec SSRE.

With a simple hassle-free integration between PureSec and Codeship, you can automate the security of your serverless projects, build, and deploy with piece of mind and confidence that your functions are robust and can withstand any type of serverless application layer attacks.

Share this Codeship Guide

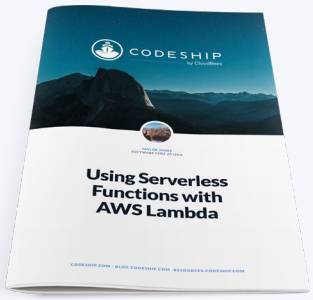
More Codeship Resources.

EBOOKS

Using Serverless Functions with AWS Lambda.

In this eBook you will learn how to execute code on third-party servers with AWS Lambda.

Download this eBook

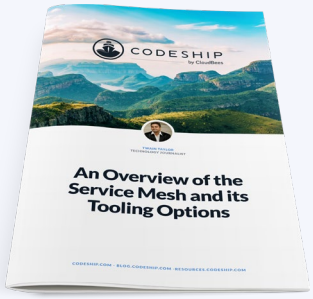


EBOOKS

An Overview of the Service Mesh and its Tooling Options.

In this eBook you will learn what a service mesh is and why it is a superior alternative to traditional networking models.

Download this eBook



EBOOKS

Deploying Docker Apps with Codeship Pro.

In this eBook we will look at building and deploying applications with Codeship Pro.

Download this eBook

CODESHIP by CloudBees

Share this Codeship Guide

About Codeship.

Codeship is a hosted Continuous Integration service that fits all your needs. Codeship Basic provides pre-installed dependencies and a simple setup UI that let you incorporate CI and CD in only minutes. Codeship Pro has native Docker support and gives you full control of your CI and CD setup while providing the convenience of a hosted solution.

Codeship Basic

A simple out-of-the-box Continuous Integration service that just works.

Starting at $0/month.

- Works out of the box
- Preinstalled CI dependencies
- Optimized hosted infrastructure
- Quick & simple setup

LEARN MORE

Codeship Pro

A fully customizable hosted Continuous Integration service.

Starting at $0/month.

- Customizability & Full Autonomy
- Local CLI tool
- Dedicated single-tenant instances
- Deploy anywhere

LEARN MORE